Universidad de las Américas Puebla

Práctica 3:
Contador ascendente–descendente de 9 bits con signo en FPGA

Profesor:
Gerardo Ulices Díaz

Integrantes:
Diego Camargo – 180892
Pablo Robles – 166855
Bruno García – 177613

Materia:
Sistemas Embebidos

Entrega:
[FECHA DE ENTREGA]

Resumen

Este reporte documenta el diseño e implementación en la tarjeta Terasic DE10-Lite (Intel MAX 10) de un contador ascendente/descendente de 9 bits con signo que recorre el intervalo completo de -256 a 255 en complemento a dos. El sistema separa una unidad de control, una FSM de tres estados (pausa, ascendente y descendente), de una ruta de datos formada por el registro del contador. Las entradas mecánicas se acondicionan con sincronizador de dos flip-flops, filtro antirrebote de 10 ms y pulso de un ciclo; la actualización del contador se rige por un pulso de habilitación (`tick`) de 1 s o 100 ms generado a partir del único reloj de 50 MHz, sin crear relojes derivados. El valor se muestra en decimal con signo en cuatro displays de 7 segmentos y el estado y la velocidad en los LEDs. La simulación y las pruebas físicas mostraron los tres estados, el cambio de velocidad y la representación de valores positivos y negativos. En el análisis, la aritmética modular explica el comportamiento en los extremos del rango, y el uso de un `tick` evita los problemas de temporización que introduce un reloj dividido. Se concluye que el diseño cumple las especificaciones; la simulación evidenció ambos cruces de los límites, $255 \to -256$ y $-256 \to 255$, y queda por documentar los recursos utilizados.

Palabras clave: FSM, complemento a dos, antirrebote, clock enable, FPGA.

Objetivos

Diseñar e implementar en Verilog un contador ascendente–descendente de 9 bits con signo, capaz de recorrer el intervalo -256 a 255, con controles de reinicio, pausa, conteo ascendente y descendente y un selector entre actualizaciones de 1 s y 100 ms, mostrando el valor en cuatro displays de 7 segmentos de la DE10-Lite.

- Separar la unidad de control (FSM) de la ruta de datos (registro con signo).
- Acondicionar los botones con sincronización, antirrebote y pulso de un ciclo.
- Generar los intervalos de 1 s y 100 ms con un pulso de habilitación en un único dominio de reloj de 50 MHz.
- Verificar el comportamiento modular en los extremos del intervalo.

Marco Teórico

Representación con signo en 9 bits

En complemento a dos, los números negativos se obtienen invirtiendo los bits de la magnitud y sumando uno, y el bit más significativo actúa como signo [3], [1]. Una palabra de $N = 9$ bits representa

$$-2^8 \le x \le 2^8 - 1 \;\Rightarrow\; -256 \le x \le 255, \qquad (1)$$

con un único cero (sin "cero negativo") [3]. Los extremos son $255 =$ `0_1111_1111`, $-1 =$ `1_1111_1111` y $-256 =$ `1_0000_0000`. Al ser aritmética modular, $255 + 1 \to -256$ y $-256 - 1 \to 255$.

Máquinas de estados finitos

Una FSM es un circuito secuencial cuyo siguiente estado depende del estado actual y de las entradas, y cuyo estado se almacena en registros actualizados con el flanco de reloj [1], [2]. Aquí se emplean tres estados: PAUSA, ASCENDENTE ($+1$ por `tick`) y DESCENDENTE (-1 por `tick`). Esta unidad de control decide *qué hacer*, mientras que la ruta de datos (el registro de 9 bits) almacena y actualiza el valor [2].

Pulso de habilitación (clock enable)

Para obtener una actualización más lenta que el reloj, es preferible generar un pulso de un ciclo (*clock enable*) y usarlo como habilitación de registros del mismo reloj, en lugar de un reloj derivado [8]. Así se mantiene un único dominio de reloj, se evitan los peligros de metastabilidad entre dominios y el análisis de temporización es más simple [8], [9]. El número de ciclos para un intervalo T con $f_{CLK} = 50$ MHz es

$$N = f_{CLK}\, T, \qquad (2)$$

es decir, $N_{1\,s} = 50{,}000{,}000$ y $N_{100\,ms} = 5{,}000{,}000$ ciclos.

Rebote y metastabilidad

Un pulsador mecánico produce varias transiciones rápidas al cerrarse (rebote), que un circuito síncrono leería como múltiples pulsaciones [5], [6]. Además, la señal del botón es asíncrona respecto al reloj; si cambia cerca del flanco puede violar los tiempos de *setup/hold* y llevar al flip-flop a un estado metaestable [4], [7]. Una cadena de dos flip-flops reduce la probabilidad de que la metastabilidad se propague, aunque no elimina el rebote [4], [7]. El filtro antirrebote exige que la señal permanezca estable un tiempo mínimo (aquí $\approx$10 ms) y se genera finalmente un pulso de un ciclo [5], [6].

La tarjeta DE10-Lite

La DE10-Lite está construida alrededor del FPGA MAX 10 10M50DAF484C7G, e incluye USB-Blaster

integrado, 10 LEDs, 10 interruptores deslizables, displays de 7 segmentos, dos pulsadores (KEY0 y KEY1) y un conector GPIO de 2×20 [10], [11].

Diseño del Experimento

Arquitectura

El sistema (Figura 1) se compone de tres módulos `button_onepulse` (pausa, incremento y decremento), un `tick_generator`, la FSM con el contador (`signed_counter_fsm`) y un decodificador a 7 segmentos con signo (`signed_display_9bit`), integrados en `counter_9bit_top`.

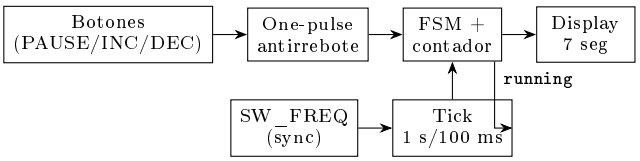


Figura 1: Diagrama a bloques del sistema.

Diagrama de estados

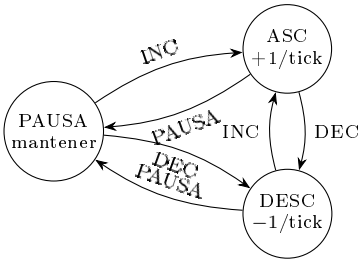


Figura 2: Estados de control. RESET lleva desde cualquier estado a PAUSA con cuenta 0.

Si dos controles coinciden en el mismo ciclo, la prioridad es PAUSA, luego INCREMENTO y finalmente DECREMENTO.

Entradas y salidas

Entrada	Función
`BTN_RESET_N`	Reinicio (cuenta 0, PAUSA), máxima prioridad
`BTN_PAUSE_N`	Pasa a PAUSA
`BTN_INC_N`	Pasa a ASCENDENTE
`BTN_DEC_N`	Pasa a DESCENDENTE
`SW_FREQ`	0: 1 s 1: 100 ms

Cuadro 1: Entradas del sistema (activas en bajo los botones).

LED	Significado
LEDR0	Estado PAUSA
LEDR1	Estado ASCENDENTE
LEDR2	Estado DESCENDENTE
LEDR3	Velocidad rápida (100 ms)
LEDR9	Bit de signo (`count[8]`)

Cuadro 2: Asignación de los LEDs de estado.

Desarrollo del Experimento

Codificación de los módulos en Verilog

El código fuente completo se encuentra en el repositorio del proyecto [13] (https://github.com/CMRGO11/Contador-9-Bits); se describe la lógica de cada bloque.

button_onepulse. Pasa la entrada por dos flip-flops y compara la muestra sincronizada con el último valor estable. Si difiere, cuenta 500,000 ciclos de 50 MHz (10 ms); si persiste, actualiza el valor estable y, solo en la transición 1→0 (pulsación), emite un pulso de un ciclo.

tick_generator. Contador de 26 bits que cuenta de 0 a $N-1$ (N = 50,000,000 o 5,000,000 según `speed_sel`) y emite un pulso de un ciclo. Se mantiene en cero cuando `enable` es 0, por lo que al reanudar transcurre un intervalo completo antes del siguiente cambio.

signed_counter_fsm. Registra el estado y el valor `count` de 9 bits con signo. Los pulsos cambian el estado con la prioridad indicada y, con cada `tick`, el contador suma o resta 1 según el estado. El reinicio asíncrono lleva el sistema a PAUSA con cuenta 0.

signed_display_9bit. Obtiene signo y magnitud, separa unidades, decenas y centenas con división y módulo entre 10 y decodifica a 7 segmentos activos en bajo. HEX3 muestra un guion si el valor es negativo; HEX2–HEX0 muestran la magnitud.

counter_9bit_top. Integra los módulos, sincroniza `SW_FREQ` con dos flip-flops y asigna los LEDs según el Cuadro 2.

Implementación en Quartus

La síntesis se realizó en Intel Quartus Prime [12] para el dispositivo MAX 10 de la DE10-Lite, con `counter_9bit_top` como entidad superior y reloj `CLOCK_50`. Como la tarjeta solo dispone de dos pulsadores integrados [10] y no se usaron pulsadores externos, únicamente el reinicio se asignó a un pulsador (KEY0) y los demás controles se asignaron a interruptores deslizables (Cuadro 3), alternativa que la propia práctica permite. Los displays HEX0–HEX3 y los LEDs integrados se asignaron a sus pines correspondientes con Pin Planner.

Señal	Recurso	Función
`BTN_RESET_N`	KEY0	Reinicio
`BTN_PAUSE_N`	SW0	Pausa
`BTN_INC_N`	SW1	Ascendente
`BTN_DEC_N`	SW2	Descendente
`SW_FREQ`	SW3	Velocidad

Cuadro 3: Asignación física utilizada en la DE10-Lite.

Al ser interruptores y no pulsadores, el módulo `button_onepulse` sigue siendo útil: sincroniza y filtra la señal y emite un único pulso por cada transición de nivel activa (1→0), de modo que mantener el interruptor en una posición no repite el comando. `SW_FREQ` se lee por nivel (0: 1 s, 1: 100 ms).

Resultados Teórico-Práctico

Resultados de la simulación funcional

La Figura 3 muestra la simulación del sistema con un `tick` abreviado para poder observar el recorrido completo del contador. En ella el sistema cuenta en estado ASCENDENTE (`state`=1), de modo que los bits de `count` conmutan en binario hasta que `count[7]` se activa a la mitad del recorrido (128 cuentas). Cerca del final de la ventana ocurren tres eventos consecutivos:

- $255 \rightarrow -256$**.** Al alcanzar 255 el siguiente `tick` activa `count[8]` y pone en cero los demás bits (`1_0000_0000` $= -256$): el contador se envuelve sin saturar.
- **Pausa y descenso.** Un `pause_pulse` lleva brevemente la FSM a PAUSA (`state`=0) y un `dec_pulse` posterior la pasa a DESCENDENTE (`state`=2).
- $-256 \rightarrow 255$**.** En descenso, el siguiente `tick` desactiva `count[8]` y activa `count[7:0]`, es decir, `0_1111_1111` $= 255$.

La señal `reset` se activa solo al final de la ventana, por lo que la simulación muestra los tres estados de control y ambas transiciones de los límites, pero no la retención del valor durante varios `tick` en pausa ni el valor posterior al reinicio. **[COMPLETAR (opcional): captura adicional de la pausa con varios ticks y del RESET llevando la cuenta a 0]**.

Tabla de pruebas

Actual	Evento	Esperado	Estado	Evidencia
0	RESET	0	PAUSA	Fig. 4a
0	INC, tick	1	ASC.	**[SIM]**
58	tick	59	ASC.	**[SIM]**
255	tick	−256	ASC.	Fig. 3
0	DEC, tick	−1	DESC.	**[SIM]**
−58	tick	−59	DESC.	**[SIM]**
−256	tick	255	DESC.	Fig. 3
45	PAUSA, ticks	45	PAUSA	Fig. 5c (en −20)
−123	RESET	0	PAUSA	**[SIM]**

Cuadro 4: Pruebas definidas por la práctica; se indica la evidencia disponible.

Resultados de la implementación física

Se descargó el diseño en la DE10-Lite y se verificó cada función. Las Figuras 4 y 5 muestran las capturas.

Reinicio y pausa. Tras el reinicio el contador queda en 000 con LEDR0 encendido (Fig. 4a). En la Fig. 5c el contador está en pausa en −20: HEX3 muestra el guion, LEDR9 indica el signo negativo y LEDR0 la pausa.

Ascenso. Con el incremento se enciende LEDR1; a velocidad lenta la captura muestra 004 (Fig. 4b) y a velocidad rápida (LEDR3 encendido) 022 (Fig. 4c).

Descenso. Con el decremento se enciende LEDR2; se muestran 040 a velocidad lenta (Fig. 5a) y 023 a velocidad rápida con LEDR3 (Fig. 5b). La cuenta continuó hacia negativos hasta −20 (Fig. 5c).

Comparación 1 s contra 100 ms. Con `SW_FREQ`=0 el contador cambia una vez por segundo (N = 50,000,000 ciclos) y con `SW_FREQ`=1 diez veces por segundo (N = 5,000,000), lo que la tarjeta indica con LEDR3. **[COMPLETAR: medición con cronómetro (p.ej. tiempo en recorrer 20 cuentas a cada velocidad)]**.

Utilización de recursos

[COMPLETAR: reporte de utilización de Quartus (elementos lógicos, registros, pines) y comentario sobre el costo de la división y el módulo del decodificador.]

Cuestionario

1. **¿Por qué 9 bits con signo representan** -256 **a 255 y no** -255 **a 255?** Hay $2^9 = 512$ patrones. Un patrón es el cero y, como el cero ocupa un patrón del lado no negativo, quedan 256 valores no negativos (0 a 255) y 256 negativos (−1 a −256). Representar −255 a 255 usaría solo 511 valores y desperdiciaría un patrón; el complemento a dos no tiene "cero negativo" [3].

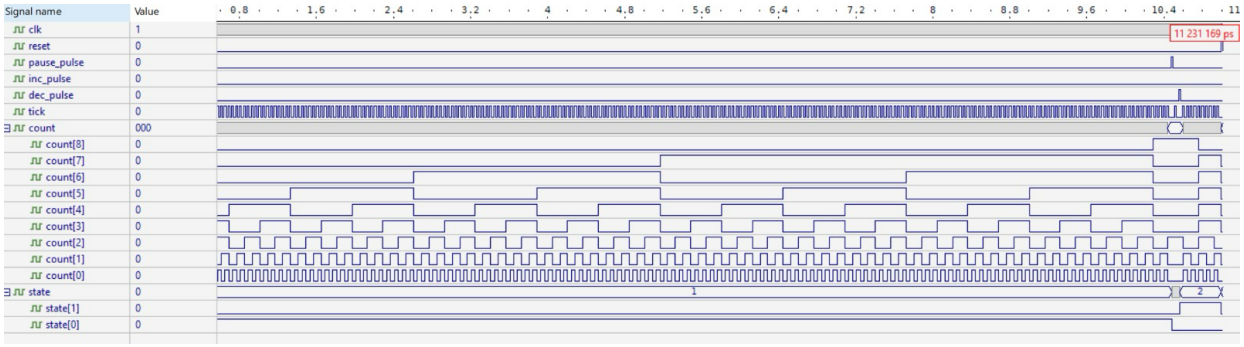
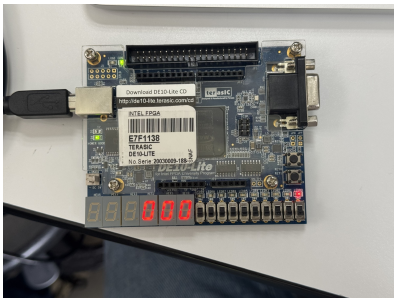
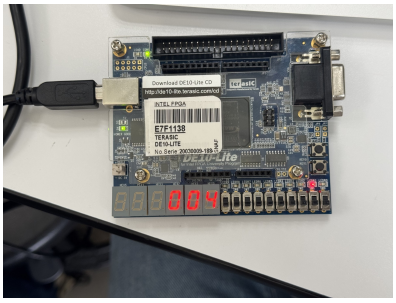


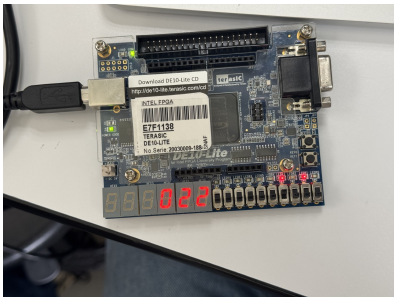
Figura 3: Simulación funcional: conteo ascendente, cruce $255 \to -256$, pausa, descenso y cruce $-256 \to 255$.



(a) Reset: 000, LEDR0.



(b) Ascenso lento: 004, LEDR1.



(c) Ascenso rápido: 022, LEDR1 y LEDR3.

Figura 4: Reinicio y ascenso en la DE10-Lite.

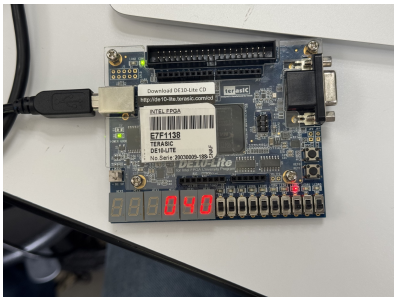
2. **¿Patrón de -256?** `1_0000_0000`.
3. **¿Qué ocurre al incrementar 255 y decrementar -256?** Por aritmética modular, `0_1111_1111` $+ 1 =$ `1_0000_0000`, es decir $255 \to -256$, y $-256 - 1 \to 255$ [3].
4. **¿Reloj nuevo o pulso de habilitación?** Un reloj nuevo crea otro dominio de reloj, con problemas de enrutamiento, temporización y cruce de dominios. El pulso de un ciclo mantiene todo en el reloj de 50 MHz y se usa como habilitación [8], [9].
5. **¿Por qué sincronizar los botones?** Son asíncronos respecto al reloj y pueden violar *setup/hold*, causando metastabilidad; dos flip-flops reducen ese riesgo [4], [7].
6. **¿Qué evita el antirrebote?** Que los rebotes mecánicos se interpreten como varias pulsaciones y el contador avance varias veces [5], [6].
7. **¿Cuántos ciclos de 50 MHz son 100 ms?** $N = (50 \times 10^6)(0.1) = 5{,}000{,}000$.
8. **¿Recursos adicionales de división y módulo?** Dividir y obtener módulo entre 10 y 100 (aunque sean constantes) se sintetiza como lógica combinacional adicional, que consume más elementos lógicos y alarga el camino crítico. **[COMPLETAR: cifras de Quartus]**.

Análisis

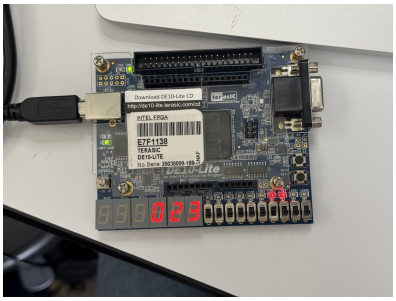
La separación entre control y datos hizo que cada módulo se verificara por separado: la FSM solo decide el modo, el registro de 9 bits aplica ± 1 y el decodificador solo interpreta el valor. El uso de un `tick` en lugar de relojes derivados mantiene un solo dominio de reloj, lo que simplifica la temporización [8], [9]. El comportamiento modular en los extremos (255 a -256 y viceversa) es consecuencia directa del ancho fijo de 9 bits: no hay saturación ni bandera de desbordamiento, a diferencia de la ALU de la Práctica 2. Finalmente, el sincronizador y el antirrebote son necesarios porque, sin ellos, una sola pulsación podría disparar varios cambios de estado [4]–[6].

Conclusión

La práctica permitió implementar un contador ascendente/descendente de 9 bits con signo controlado por una FSM en la DE10-Lite. Se explicó la representación en complemento a dos y su comportamiento modular, la separación entre unidad de control y ruta de datos, la función del pulso de habilitación y el acondicionamiento de entradas. Las capturas físicas confirmaron los estados de pausa, ascenso y descenso, el cambio de velocidad y la visualización de valores



(a) Descenso lento: 040, LEDR2.



(b) Descenso rápido: 023, LEDR2 y LEDR3.



(c) Pausa en −20: LEDR9 y LEDR0.

Figura 5: Descenso, cambio de velocidad y pausa con valor negativo.

negativos con el guion y LEDR9. Es preferible mantener un único reloj y un `tick` porque evita dominios de reloj adicionales, metastabilidad entre ellos y problemas de cierre de temporización [8], [9]. **[AÑADIR: conclusiones propias del equipo]**.

References

[1] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed. Pearson, 2018.

[2] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed. McGraw-Hill, 2014.

[3] "Two's complement," Wikipedia. [En línea]. Disponible: https://en.wikipedia.org/wiki/Two%27s_complement. [Consultado: 16-oct-2026].

[4] "Metastability and Clock Domain Crossing," notas de clase, Dept. of Electrical and Computer Engineering, Univ. of British Columbia. [En línea]. Disponible: https://people.ece.ubc.ca/~edc/7660.jan2017/lectures/lec11.pdf. [Consultado: 16-oct-2026].

[5] L. Llamas, "FPGA Debounce: How to Read Push Buttons Correctly." [En línea]. Disponible: https://www.luisllamas.es/en/fpga-debounce-eliminar-rebotes/. [Consultado: 16-oct-2026].

[6] "What is Debouncing in FPGA?," DEV Community. [En línea]. Disponible: https://dev.to/carolineee/what-is-debouncing-in-fpga-40ke. [Consultado: 16-oct-2026].

[7] "Metastability," FPGARelated.com. [En línea]. Disponible: https://www.fpgarelated.com/glossary/metastability. [Consultado: 16-oct-2026].

[8] "Clock Enable," FPGARelated.com. [En línea]. Disponible: https://www.fpgarelated.com/glossary/clock-enable. [Consultado: 16-oct-2026].

[9] "CPLD Clock divider?," Intel/Altera Community Forum. [En línea]. Disponible: https://community.intel.com/t5/Programmable-Devices/CPLD-CLock-divider/td-p/7533. [Consultado: 16-oct-2026].

[10] Terasic Technologies, *DE10-Lite User Manual*. [En línea]. Disponible: https://eecs.engineering.oregonstate.edu/education/inventory_datasheets/P1533579709.pdf. [Consultado: 16-oct-2026].

[11] "Terasic Technologies DE10-Lite Board," Mouser Electronics. [En línea]. Disponible: https://www.mouser.ca/terasic-de10-lite-board. [Consultado: 16-oct-2026].

[12] Intel, *Quartus Prime Design Software Documentation*.

[13] D. Camargo, P. Robles y B. García, "Contador-9-Bits: contador ascendente–descendente de 9 bits con signo en DE10-Lite," repositorio GitHub. [En línea]. Disponible: https://github.com/CMRGO11/Contador-9-Bits. [Consultado: 20-oct-2026].